

PORTADA

Análisis y Diseño de Sistemas II

**Semana 9 — SOLID: Dependency Inversion (DIP) y Laboratorio de cierre de
Unidad 4**

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**Cambiar de base de datos
no debería obligar a reescribir la lógica de negocio**

TRANSICIÓN

El quinto y último principio SOLID

El martes vimos **ISP**: interfaces pequeñas, una por rol de cliente. Hoy vemos el principio que completa el acrónimo:

Dependency Inversion Principle (DIP) — de qué depende el código de alto nivel y por qué eso importa.

Con DIP se cierran los cinco principios SOLID y la **Unidad 4**. La segunda parte de hoy es el **laboratorio de cierre**, donde cada equipo aplica los cinco principios completos sobre su propio proyecto.

CONTRAEJEMPLO

Alto nivel creando su propio detalle de bajo nivel

```
class ServicioDeNotificaciones:
    def __init__(self):
        self.smtp = ClienteSntp()    # instancia concreta, con "new"

    def notificar(self, mensaje):
        self.smtp.enviar(mensaje)
```

`ServicioDeNotificaciones` es lógica de negocio de **alto nivel**: decide cuándo notificar. `ClienteSntp` es un detalle técnico de **bajo nivel**: cómo se manda un correo. Al crearlo con `ClienteSntp()` directamente, la clase de alto nivel queda **atada** a esa implementación concreta.

CONCEPTO

El problema: depender del detalle

Si el equipo decide cambiar de correo (SMTP) a un servicio de mensajería push, hay que **editar** `ServicioDeNotificaciones` — aunque su lógica de negocio (cuándo notificar) no cambió en nada. El detalle de bajo nivel terminó dictando cambios en el módulo de alto nivel.

Además, para probar `ServicioDeNotificaciones` con una prueba unitaria hay que enviar un correo real, porque no hay forma de reemplazar `ClienteSmtplib` por una versión de prueba — la dependencia está fija dentro del código.

CONCEPTO

Definición formal de DIP

Dependency Inversion Principle (Robert C. Martin) tiene dos partes:

- Los módulos de **alto nivel** no deben depender de módulos de **bajo nivel**. Ambos deben depender de **abstracciones**.
- Las abstracciones no deben depender de detalles. Los **detalles** deben depender de las abstracciones.

Una **abstracción**, en este contexto, es una interfaz o clase abstracta que define un contrato sin decir cómo se implementa. La "inversión" está en que, normalmente, el bajo nivel dictaría el diseño del alto nivel — DIP invierte esa dirección: ambos apuntan hacia el mismo contrato.

CONCEPTO

La solución: depender de una interfaz

```
interface CanalDeNotificacion:
    def enviar(self, mensaje): ...

class ClienteSmtplib implements CanalDeNotificacion:
    def enviar(self, mensaje): ... # detalle: correo

class ServicioDeNotificaciones:
    def __init__(self, canal: CanalDeNotificacion):
        self.canal = canal          # abstracción inyectada

    def notificar(self, mensaje):
        self.canal.enviar(mensaje)
```

Ahora `ServicioDeNotificaciones` depende de `CanalDeNotificacion` (abstracción), no de `ClienteSmtplib` (detalle). El objeto concreto se le entrega desde afuera, en el constructor.

CONCEPTO · ACLARACIÓN IMPORTANTE

DIP no es lo mismo que "inyección de dependencias"

Es un error común usar los dos términos como sinónimos. No lo son:

- **DIP** es un **principio de diseño**: dice hacia dónde deben apuntar las dependencias (hacia abstracciones).
- **Inyección de Dependencias (Dependency Injection, DI)** es una **técnica**: la forma mecánica de entregarle a una clase el objeto concreto que necesita (por constructor, como en el ejemplo anterior, por método, o mediante un framework/contenedor de DI), en vez de que la clase lo cree con `new`.

DI es **una** manera de lograr DIP en la práctica — la más común — pero se puede cumplir DIP sin un framework de DI, y se puede usar DI para inyectar igual una clase concreta (sin abstracción), lo cual no resuelve DIP.

CONCEPTO

Qué se gana al invertir la dependencia

- **Se puede cambiar el detalle sin tocar el alto nivel:** pasar de `ClienteSmtp` a un `ClientePush` nuevo solo exige que implemente `CanalDeNotificacion`.
- **Pruebas unitarias reales:** se puede inyectar un `CanalDeNotificacion` de prueba (un "doble" que solo registra que se llamó) sin enviar correos de verdad.
- **El módulo de negocio no conoce detalles técnicos:** no sabe si detrás hay SMTP, una API o una cola de mensajes — solo conoce el contrato.

CONCEPTO · RELACIÓN CON LO VISTO

DIP es el porqué detrás de las arquitecturas N-capas

En la semana 6 vimos el patrón **Layers**: presentación, lógica de negocio y persistencia separadas en capas, donde cada capa superior depende de la inferior. DIP explica **por qué** esa dependencia es riesgosa si se deja tal cual: la capa de dominio (alto nivel, las reglas del negocio) terminaría dependiendo directamente de la capa de persistencia (bajo nivel, el detalle de cómo se guardan los datos).

Aplicando DIP, la capa de dominio define una **interfaz de repositorio** (la abstracción) y la capa de persistencia la implementa — el dominio no depende de la base de datos, la base de datos depende del contrato que el dominio definió.

CONCEPTO · PREVIEW

Un nombre que van a encontrar más adelante

Llevar DIP al extremo de una arquitectura completa —donde el dominio no depende de nada externo y todo lo externo (base de datos, UI, servicios de terceros) depende de interfaces que el dominio define— es la idea central de estilos como **Clean Architecture** o **Ports and Adapters** (arquitectura hexagonal).

No los desarrollamos en este curso, pero ahora tienen el principio de base (DIP) para entenderlos cuando los encuentren en un curso posterior o en un proyecto profesional.

TENSIÓN CONCEPTUAL

DIP no significa "una interfaz para cada clase"

Igual que con OCP y ISP, aplicar DIP a ciegas —creando una interfaz para absolutamente todo, incluidas clases simples que jamás van a cambiar de implementación— agrega indirección sin beneficio real y hace el código más difícil de seguir.

DIP se justifica en los puntos donde el detalle realmente puede cambiar o donde se necesita aislar para probar: acceso a datos, servicios externos, el sistema de archivos, el reloj del sistema. Una clase de utilidad interna sin variación esperada no necesita una interfaz solo por principio.



INTERACCIÓN

¿Dónde sí vale la pena invertir la dependencia?

En el chat, para cada caso decidan si conviene aplicar DIP (definir una abstracción e inyectar la implementación) o si el detalle no lo amerita:

1. Una clase de dominio que hoy guarda datos directo con una librería específica de base de datos, y el equipo no descarta cambiar de motor.
2. Una función interna que solo formatea texto y no tiene ninguna variación prevista.
3. Un módulo que hoy consume una API de pagos externa, y el negocio pide poder soportar un segundo proveedor.

5 minutos · respondan en el chat

SÍNTESIS

DIP, en una frase

1. **DIP**: alto nivel y bajo nivel dependen de abstracciones, no uno del otro; las abstracciones no dependen de detalles.
2. **DIP $\neq$ Inyección de Dependencias**: DI es la técnica más común para lograr DIP, no su sinónimo.
3. Es el principio detrás de por qué el dominio no debería depender de la infraestructura en una arquitectura por capas (semana 6).
4. Se justifica en puntos de variación real o necesidad de prueba — no en cada clase del sistema.



Los cinco principios SOLID, completos

Letra	Principio	Idea central
S	Single Responsibility	Una clase, una razón para cambiar
O	Open/Closed	Abierta a extensión, cerrada a modificación
L	Liskov Substitution	Toda subclase debe poder sustituir a su superclase sin romper nada
I	Interface Segregation	Ningún cliente debe depender de métodos que no usa
D	Dependency Inversion	Ambos niveles dependen de abstracciones, no el uno del otro

Con esto se completan las tres semanas de SOLID: semana 7 (fundamentos de POO), semana 8 (S, O, L) y semana 9 (I, D).

TRANSICIÓN

Ahora, a aplicarlo sobre el proyecto real

Los cinco principios se vieron con ejemplos de clase. El resto de la sesión es el **laboratorio de cierre de la Unidad 4**: cada equipo revisa su propio diagrama de clases y código del proyecto de curso, buscando dónde SOLID ya se cumple y dónde no.

INSTRUCCIONES

Laboratorio: aplicación de SOLID al proyecto

Laboratorio de cierre de Unidad 4 — en equipo, con el mismo equipo del proyecto de Fábrica Escuela conformado desde la semana 2.

- Retomen el **diagrama de clases** y el código actual de su proyecto (construido desde la semana 2).
- Para **cada uno de los cinco principios SOLID**, identifiquen al menos una violación real o potencial en su propio diseño — o, si genuinamente no aplica ningún caso, justifiquen por qué.
- Para cada violación identificada, propongan **cómo la corregirían**: qué clase, interfaz o dependencia cambiaría y hacia qué diseño.

Durante el resto de la sesión · trabajo en equipo sobre el proyecto propio

INSTRUCCIONES

Entregable y cierre

- **Formato exacto:** un breve análisis SOLID del diagrama de clases actual, documentado en el repositorio o herramienta de gestión del proyecto del equipo.
- Debe señalar, **por cada uno de los cinco principios**, al menos una violación identificada y corregida — o la justificación explícita de por qué ese principio no aplica en su caso.
- Este análisis es la **preparación directa** para el hito de evaluación de esta unidad.

Momento 1 (20%) — Diagrama de Clases y aplicación de Principios básicos de Diseño, se evalúa el **miércoles 30/09**. El análisis SOLID de hoy es el insumo base con el que cada equipo llega a esa evaluación.



CIERRE

Cierre de la Unidad 4

Con esto se cierra la Unidad 4 — Principios Básicos de Diseño. En tres semanas se cubrieron los fundamentos de POO (abstracción, encapsulamiento, cohesión, acoplamiento, herencia, polimorfismo) y los cinco principios SOLID completos, aplicados al proyecto propio.

El miércoles 30/09 es el **Momento 1 (20%)**: presenten su diagrama de clases junto con el análisis SOLID de hoy, mostrando las violaciones identificadas y cómo las corrigieron.

